



PUC Minas

Plataformas Node.js

Samuel Martins



PUC Minas

Arquitetura CLEAN

Arquitetura CLEAN

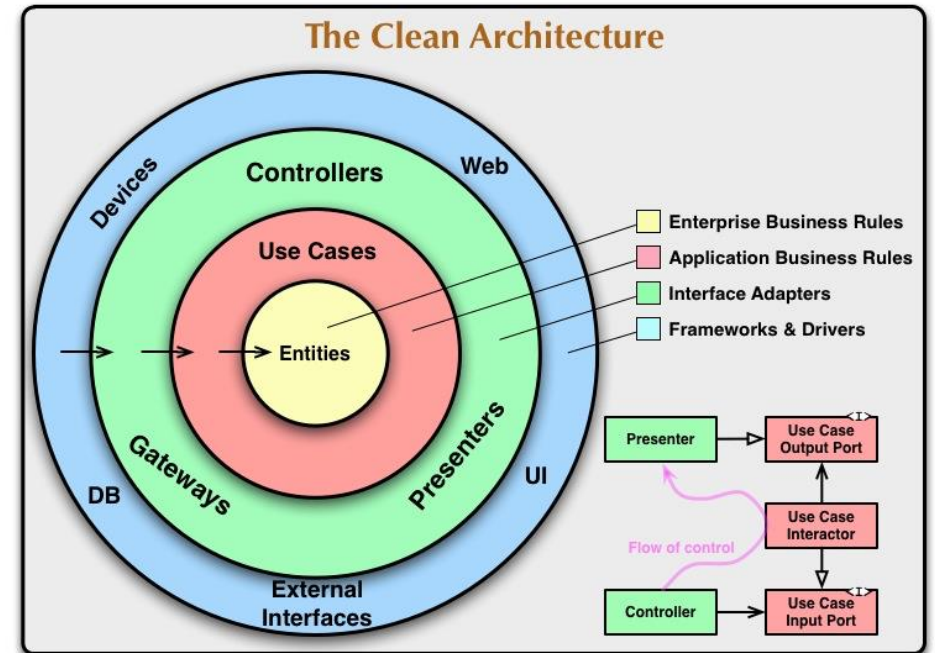
- **Arquitetura Clean** (ou simplesmente Limpa) é uma tentativa de criar um modelo que ofereça separação de responsabilidades através de camadas;
- Combinação com uma série de melhorias de arquiteturas criadas anteriormente, como a Hexagonal (ou Ports and Adapters);

Arquitetura CLEAN

- Dentre os pilares desses modelos arquiteturais, podemos citar:
 - **Independência de frameworks:** arquitetura pode ser aplicada em qualquer framework de qualquer linguagem;
 - **Testabilidade:** regras de negócio devem poder ser testadas sem depender da camada de UI;
 - **Independência da UI:** a interface deve poder ser desacoplada e independente da regra de negócio da aplicação e dos casos de uso;
 - **Independência de terceiros:** regras de negócio específicas da aplicação não devem conhecer sobre interfaces do mundo externo.

Arquitetura CLEAN

- A regra primordial para o funcionamento dessa arquitetura é a regra da dependência: os elementos de um círculo interno não devem saber nada sobre os elementos de um círculo externo;
- Resumidamente, algo declarado em um círculo externo não deve ser mencionado diretamente pelo código em um círculo interno.



Fonte: <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>



PUC Minas

Persistência de dados

Bancos de dados relacionais (SQL)



SQLite

- Banco de dados embarcado na aplicação - o arquivo com os dados fica junto com o código-fonte;
- Nenhuma configuração necessária: basta criar um arquivo e pronto;
- Utiliza linguagem SQL com a grande maioria das principais instruções e é compatível com todos os sistemas operacionais;



Fonte: <https://www.sqlite.org>

SQLite

- Ideal para uso de aplicações de teste ou aplicações de pequeno porte em geral;
- A grande vantagem do SQLite sobre outras opções como o MySQL é a facilidade de integração com aplicações de qualquer linguagem e a possibilidade de desenvolver uma aplicação sem configurações complexas de banco de dados.



Fonte: <https://www.sqlite.org>



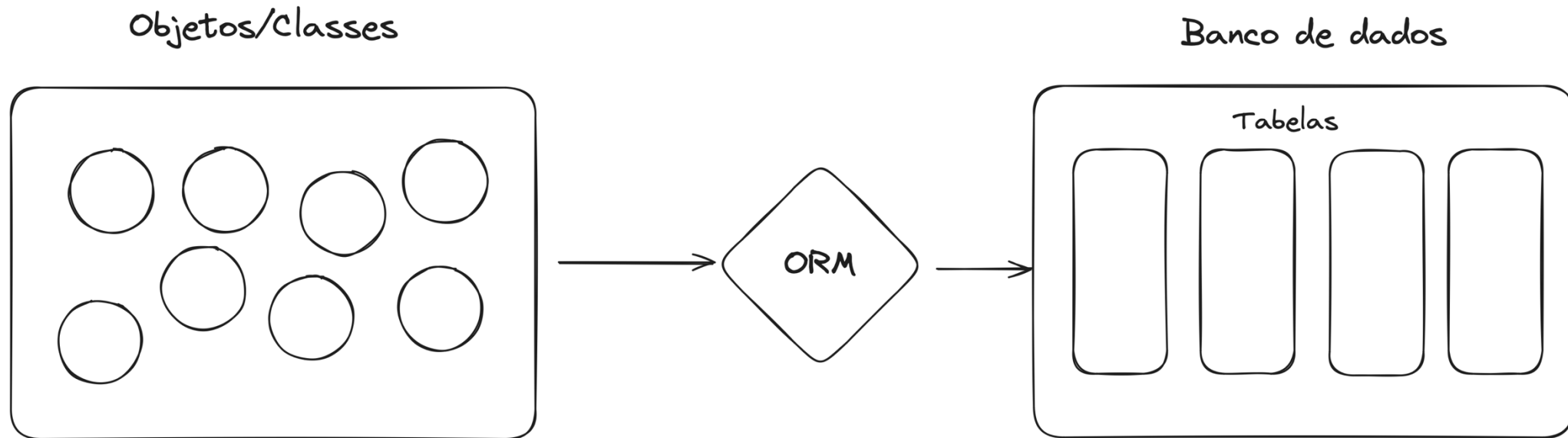
PUC Minas

ORM: Object Relational Mapping

ORM: Object Relational Mapping

- Técnica de meta-programação que tem como objetivo transformar classes/objetos em tabelas/colunas de banco de dados;
- A representação do banco é espelhada em Classes e propriedades, facilitando a manutenção do esquema de dados e a sua utilização no código-fonte;
- Abstrai a implementação da integração entre linguagem e banco de dados: permite a quem está desenvolvendo a utilizar métodos abstratos que representam as instruções SQL;
- Portabilidade: possível portar entre diferentes tipos de bancos de dados sem a necessidade de refatorar toda a aplicação.

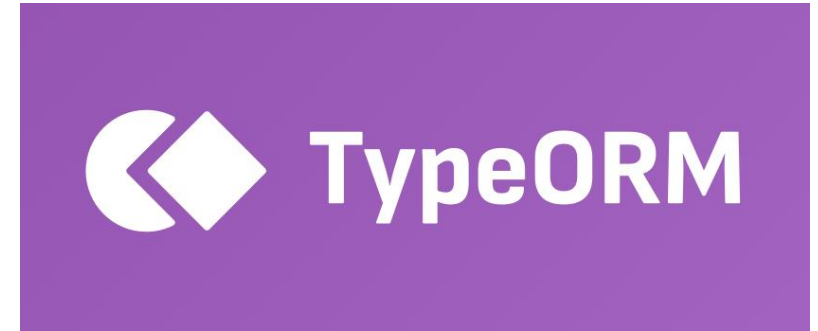
ORM: Object Relational Mapping



Fonte: próprio autor

TypeORM

- O TypeORM é uma biblioteca escrita em TypeScript que implementa o conceito de ORM para aplicações baseadas em Nodejs;
- Pode ser utilizada com qualquer framework e tem integração facilitada com o NestJs;
- Possui um conjunto de validadores e uma série métodos para acessar o conteúdo do banco de dados



Fonte: <https://typeorm.io>

TypeORM

```
import { Module } from "@nestjs/common"
import { TypeOrmModule } from "@nestjs/typeorm"

@Module({
  imports: [
    TypeOrmModule.forRoot({
      type: "sqlite",
      database: "db/sql.sqlite",
      synchronize: true,
      autoLoadEntities: true,
    }),
  ],
})
export class TypeOrmConfig {}
```



PUC Minas

Entidades

Entidades

- Entidades fazem o papel de implementar os objetos que serão mapeados para o banco de dados. Podemos colocar as propriedades, seus tipos e as validações necessárias para que elas sejam convertidas em tabelas no SQL.

```
import { Column, Entity, PrimaryGeneratedColumn }  
from "typeorm"  
  
@Entity()  
export class Project {  
  @PrimaryGeneratedColumn()  
  id: number  
  
  @Column({ name: "name", nullable: false })  
  name: string  
  
  @Column({ name: "description", nullable: false })  
  description: string  
}
```

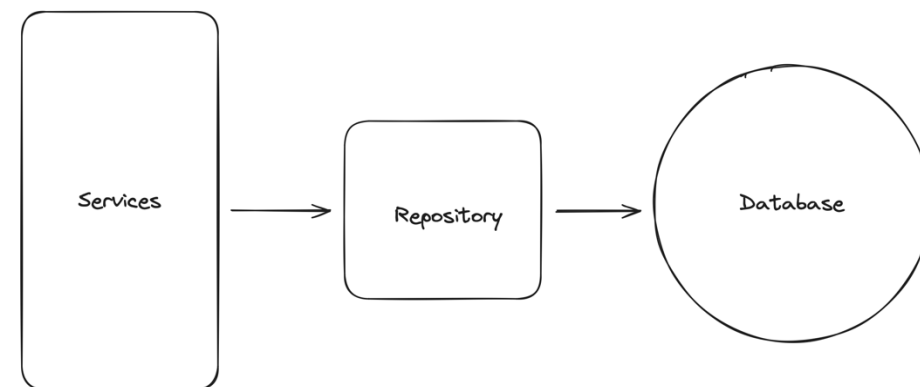



PUC Minas

Padrão repositório

Padrão repositório

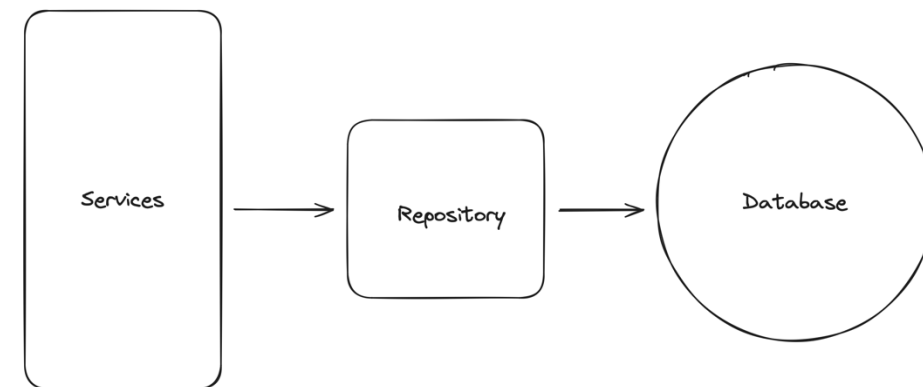
- Visa separar a lógica de persistência da camada do domínio de negócio;
- É uma camada que fica entre o Serviço e o banco de dados;
- A implementação de queries e toda a logica direta de acesso a dados fica na camada repositor;
- A grande vantagem é que qualquer alteração na camada de dados não afeta a implementação da regra de negócio.



Fonte: própria do autor

Padrão repositório

- A grande vantagem é que qualquer alteração na camada de dados não afeta a implementação da regra de negócio.



Fonte: própria do autor



PUC Minas

Padrão DTO – Data Transfer Object

Padrão DTO

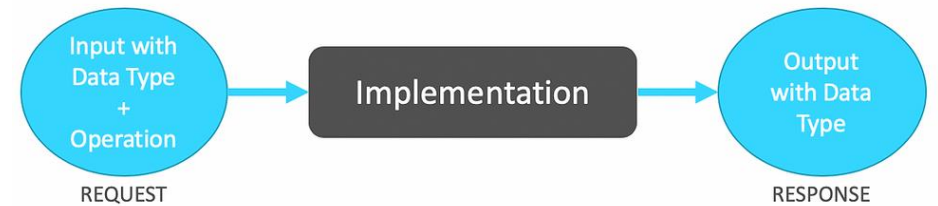
- Uma vez que temos as entidades prontas para serem mapeadas para o banco de dados, podemos criar um DTO, que serve para mapear os dados da entidade para o cliente;
- Teoricamente, poderíamos expor essas entidades diretamente para os serviços, entretanto nem sempre é uma boa ideia transferir toda a entidade para o cliente, visto que:
- Eventualmente, precisamos ocultar propriedades específicas;
- O cliente pode enviar dados diferentes dos que são armazenados no banco de dados para executar alguma regra de negócio específica;

Padrão DTO

- Evita vulnerabilidades de excesso de postagem - quando enviamos dados não desejados do banco para o cliente;
- Maior desacoplamento entre a camada de serviço e a camada de banco de dados;

Padrão DTO

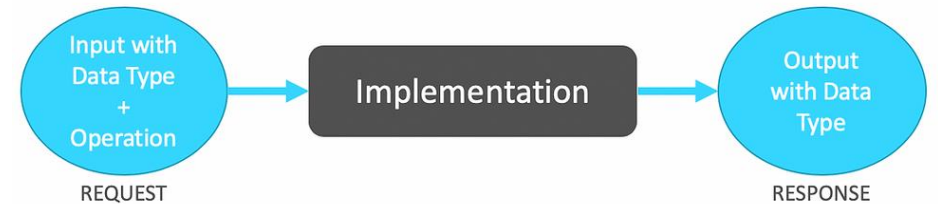
- Podemos implementar técnicas de validações que nos garantem uma API mais segura e confiável no tratamento de dados;
- Input e Output de dados podem ser customizados globalmente através de validadores e definição dos tipos via DTO;



Fonte: <https://www.prostdev.com/post/understanding-apis-part-1-what-is-an-api>

Padrão DTO

- As validações de Input garantem que os dados que chegam via controladores estão de acordo com o que definimos previamente;
- As validações de Output garantem que estamos expondo apenas os dados necessários como resposta a uma requisição.



Fonte: <https://www.prostdev.com/post/understanding-apis-part-1-what-is-an-api>



PUC Minas

CQRS: Command and Query Responsibility Segregation

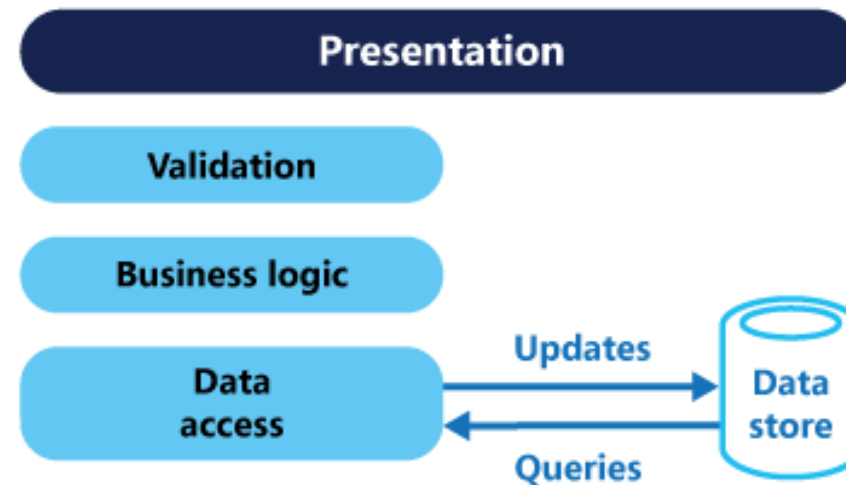
- O fluxo de operações de aplicações CRUD (Create, Read, Updated e Delete) pode ser descrito nos seguintes passos:
- A camada de controller gerencia as requisições HTTP e delega tarefas para a camada de serviços;
- A camada de serviços é onde a maior parte da regra de negócio é implementada;
- Serviços usam repositórios para acessar/alterar dados persistidos;
- Entidades performam como containers para valores e get/set.

CQRS

- Esse fluxo normalmente é mais do que necessário para aplicações pequenas/médias, mas pode não ser a melhor abordagem para aplicações em larga escala;
- Para grandes aplicações, o problema do fluxo anterior começa a aparecer quando precisamos de, eventualmente, customizar de formas diferentes as operações de escrita e leitura;
- Operações de escrita e leitura podem precisar de estruturas de dados diferentes para otimização de performance e diferentes dimensionamentos;

CQRS

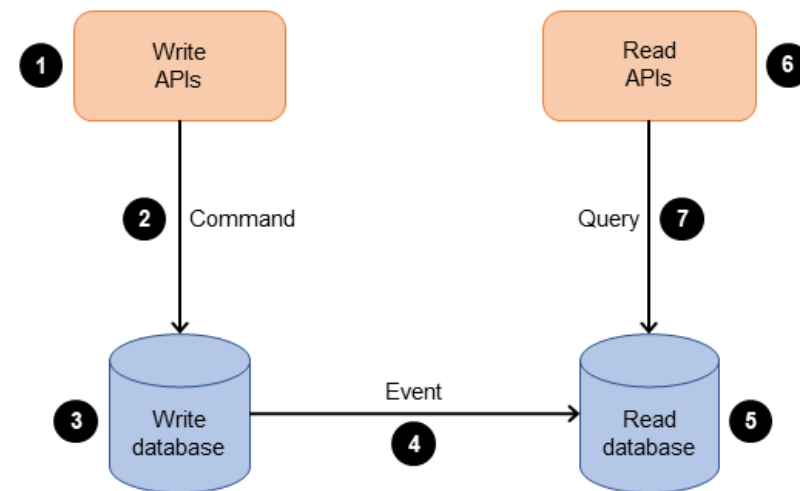
- Além disso, as representações de leitura e gravação de dados geralmente são diferentes (campos necessários durante atualizações podem ser desnecessários durante as leituras)



Fonte: <https://learn.microsoft.com/pt-br/azure/architecture/patterns/cqrs>

CQRS

- O CQRS tenta sanar problemas de:
 - **Separação de responsabilidades:** Separa operações de leitura e escrita em modelos separados;
 - **Escalabilidade:** operações de leitura e escrita são escaladas de forma independente, se necessário;
 - **Flexibilidade:** utilização de tipos diferentes de armazenamentos para leitura e escrita.



Fonte: https://docs.aws.amazon.com/pt_br/prescriptive-guidance/latest/modernization-data-persistence/cqrs-pattern.html



PUC Minas

Autenticação

Autenticação

- Aplicações que requerem uma área logada precisam de um sistema sólido de autenticação;
- A aplicação deve reagir conforme o usuário logado (operações de leitura e escrita);
- A autenticação pode ser feita de inúmeras formas: tokens, sessões...



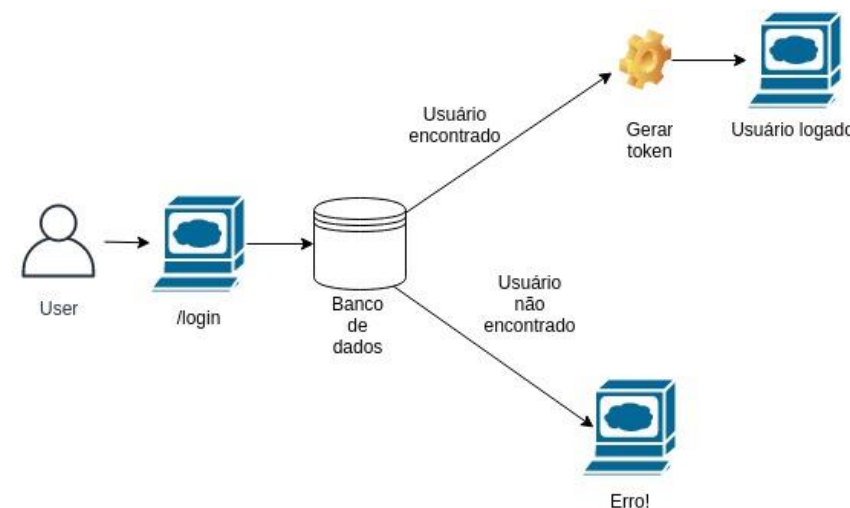
Fonte: <https://securityintelligence.com/why-you-need-advanced-authentication-to-protect-identities-without-compromising-user-experience/>

Autenticação – Token JWT

- JWT (JSON Web Token) é um padrão aberto ([RFC 7519](#)) que define uma maneira compacta e segura de transmitir informações entre partes como um objeto JSON;
- É comumente usado para autenticação e autorização em aplicativos web e APIs;
- Permite que os usuários se identifiquem de forma segura e confiável sem a necessidade de armazenar informações de sessão no servidor.

Autenticação – Token JWT

- O usuário envia credenciais de login para o servidor.
- O servidor autentica o usuário e gera um JWT contendo informações de identificação e envia para o cliente;
- O cliente armazena o JWT (em local storage ou cookies) e o envia no cabeçalho de todas as solicitações subsequentes ao servidor.



Fonte: própria do autor

Autenticação – Token JWT

- Bearer authentication (ou token authentication) é um [esquema de autenticação HTTP](#) que é gerada de forma encriptada pelo servidor e enviada ao cliente;
- O cliente precisa de ter um cabeçalho chamado Authorization com a seguinte estrutura:
 - **Authorization: Bearer <token>**
- O nestjs fornece dois recursos para gerenciamento de autenticações: AuthGuards e JwtService;



PUC Minas